

Data Visualization and Exploration

COMS7066A/COMS4060A

Muhammad Nasir

Lecture 7



Lesson Plan

- Linear dimensionality reduction this week
 - Principal Component Analysis
 - Factor Analysis
 - Independent Component Analysis
 - Linear Discriminant Analysis

Dimensionality reduction

- We've mentioned multiple times how important dimensionality reduction can be when doing feature engineering, particularly for large datasets:
- High dimensional data is difficult to interpret – if it's in 2 or 3 dimensions it is easy for us, as humans, to visualise
- It is potentially expensive to do computations with many features
- It is easy to overfit models
- Obviously, we could just drop random variables and see what happens, but we might leave out important information! There must be a better way.
- Really, we want to keep as much of the information as possible, but chuck out the junk

Feature engineering, elimination and dimensionality reduction

- Feature elimination – removing explicit features. Sometimes obvious, but it can easily lose information those dropped variables would bring.
- Feature engineering – creation of new (hopefully) independent variables
 - Again, we might lose some information, or we might not necessarily know which of the old variables are the most important.
- The key part to dimensionality reduction is that we only keep a subset of our newly engineered or extracted features, ranked according to their predictive power.
- However, these new variables we are using are combinations of our old variables, so when we keep some of these new variables, we get to keep the most valuable parts of all the old variables!

Some basics we need to cover

- **Variance**
 - This gives us an idea of how spread out the data is – the higher the variance, the more spread out the data is. It can be negative (value of zero means every data point is the same).
- **Covariance**
 - How much one variable changes as another changes – positive means they show similar behaviour, negative is opposite. Note that the magnitude is not easy to interpret – you should be looking at the correlation coefficient for this.
 - If one variable is highly correlated with another, then we can exclude one without losing too much
- **Covariance matrix:**
 - Square matrix with rows and columns for each feature in the data, and each entry showing the covariance between the corresponding feature in the row and column
- **Independence:**
 - Variables are independent if they do not influence each other. You can use statistical test to check for independence
- **Eigenvectors and eigenvalues**
 - Basically, if you have a matrix A , then v is an eigenvector of A if $Av = \lambda v$ for some scalar value of λ . That's pretty much all you need to remember
- **Linear separability**
 - If we're only talking about 2D, then this is easy to understand. If we have a dataset with two classes, can we draw a straight line that completely separates them? • In higher dimensions we talk about hyperplanes instead of lines, but it is the same idea

Linearity

- So we're looking at linear dimensionality reduction ... How do we know if our data is even linearly separable?!
- Unfortunately, there is no simple answer to this question
 - If there are relatively few features/classes in the data, say, 2, then this is easy to do by just plotting it and seeing for yourself that it is so
 - If we're working in much higher dimensions, this procedure won't work...
- Some ideas to try:
 - SVM: If we have labelled data and are working on a classification task, support vector machines can be useful. Specifically, these separate data into distinct hyperplanes, which is exactly what we're looking for
 - Logistic regression: if you have low dimensionality data you can do this and check how good the accuracy is
 - Simple perceptron: Simple perceptrons will converge if the data is linear, but obviously this does not work in higher dimensions
 - Clustering: If you have relatively few features (say <10), then clustering might be useful too



Linearity checks

- We can build a basic classifier in just a few lines in Python and run our data through the classifier. If there is any sort of linearity in there, then the basic model should pick it up and not do terribly...
- Plugging the iris dataset into `check_linearity()` returns an accuracy of 94% - I'd say that it is linear
- Similarly, the `check_linearity_svm()` method called on iris and the wine dataset scored even higher
- This is a good sanity check to do first, BUT, you can just start with something like PCA and see what the results look like

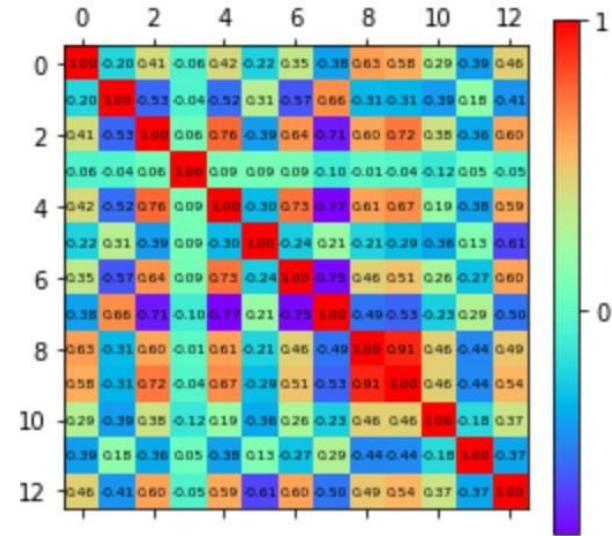
```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

# from sklearn
def check_linearity(X, Y):
    X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.
25, random_state=42)
    reg = LinearRegression().fit(X_train, y_train)
    return r2_score(reg.predict(X_test), y_test)

def check_linearity_svm(X, Y):
    X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.
25, random_state=42)
    clf = SVC()
    scaler = StandardScaler()
    clf.fit(scaler.fit_transform(X_train), y_train)
    return clf.score(scaler.transform(X_test), y_test)
```

Correlation And Covariance

- Correlation and covariance are not the same thing! Almost though... Up to a scaling factor
- Covariance is not normalized, so interpreting a value is not meaningful by itself, it is only when compared with other values
- Covariance matrix (ie, the covariance between elements of some matrix X) is given by:
$$\text{cov}[X, X] = E[XX^T] - E[X]E[X^T]$$
, but we will often abbreviate this to XX^T
- Correlation DOES NOT imply causation
- This is important - if two variables are independent, that implies that there is no correlation. However, no correlation does not imply that the variables are independent.
- Types of correlation:
 - Pearson's for linear correlation
 - Spearman's for non-linear



Each square represents the correlation between the variable in that row and that column.

Extreme values (red and purple) indicate a high (negative) correlation

Eigendecomposition & SVD

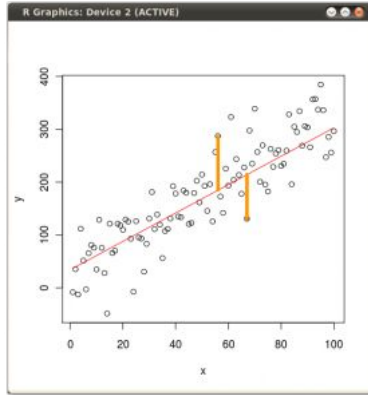
- This refers to the factorization of a matrix into its eigenvalues and eigenvectors – basically, we want to take a matrix and represent it (or use some of its properties) slightly differently to make life easier.
- Basically if we have some square matrix A , then its eigenvector v will satisfy
 - $Av = \lambda v$ where λ is the eigenvalue.
- Eigenvectors get transformed by A , and the eigenvalue says how much it stretches or shrinks the eigenvector. So, typically, a larger λ means that eigenvector component has a larger impact on A .
- What does this mean? This all sounds very mathsy... it is, but you only need to really focus on some of the outcomes of it, and what the eigenvalues and vectors mean in specific circumstances
- Similarly, singular value decomposition also works by breaking down a matrix A into separate pieces, but it doesn't need to be diagonalizable, as it does for eigendecomposition
 - $A = U\sigma V$, where σ is a diagonal matrix with the entries being equal to the square root of the eigenvalues of A .

PCA - Overview

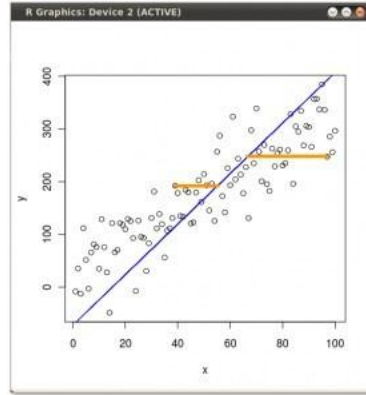
- The information contained in a feature is the amount of variance it contains, or can explain.
 - If we can find a subset of features that contain most of the variance, we can capture most of the underlying information even after discarding the other features.
- This is exactly what PCA does - it is essentially looking for a linear combination of variables able to explain the most variance in the dataset.
 - More formally, PCA is a projection based method which transforms data into a lower dimensional space.
- The principal components are those linear combinations that contain the maximum amount of variance. The first principal component explains maximum variance in the dataset, the second principal component tries to explain the remaining variance in the dataset and is uncorrelated to the first principal component. The third explains the remaining variance and so on. Each principal component has a magnitude and direction, where the direction represents along which principal axes the data has the most variance and the magnitude is the amount of variance that principal component captures of the data when projected onto that axis.
- PCA is an unsupervised method - it identifies the combination of attributes (principal components, or directions in the feature space) that account for the most variance in the data, regardless of any classes or labels.
- Choosing the number of principal components will depend on the application - if you want to visualise the data, you want 2 or 3, if you are using the PCs as input to a subsequent algorithm, then you might want to increase this so that you can capture more of the variance.
 - Use a scree plot to see how much variance is being captured by the components
- Finally, we make an assumption that more variability in a particular direction correlates with explaining the behaviour of the dependent variable. Lots of variability usually indicates signal, whereas little variability usually indicates noise. Thus, the more variability there is in a particular direction is, theoretically, indicative of something important we want to detect.
- Before applying PCA, your data must be standardised! PCA is trying to find the components that maximise the variance, but if two components are on completely different scales, this can significantly skew the results. For example: we have features of weight (in kg) and height (in metres). If we do not standardise the data, then PCA will likely determine that the majority of the variance is held by weight, purely because it varies between, say, 40kg and 100kg (up to 60 kg difference!), and height only varies from 1.4m to 1.9m (so .5).

PCA - Overview

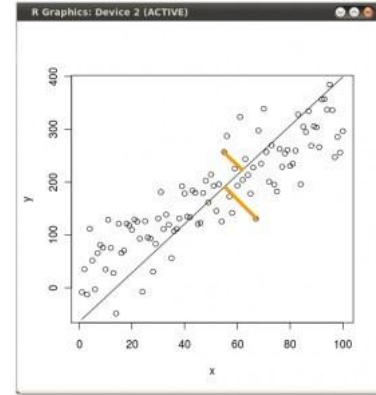
Y regressed against x



X regressed against Y



PCA – orthogonal to line of best fit



PCA - Overview

- We take our original matrix and standardise it, X , and, up to a constant, we have the covariance matrix $X^T X$, we decompose this into PDP^{-1} , where P is the matrix of eigenvectors, D is the diagonal matrix of eigenvalues. We then sort the eigenvalues λ_i in descending order (and P at the same time). We can call the sorted eigenvector matrix P^* . Now, we can calculate the principle components of the data by taking the dot product of X and P^* , according to $X^* = XP^*$. Every entry in here is now a combination of original (standardised) variables, weighted by the weights in P^* .
- There is another, almost equivalent way of doing this, via singular value decomposition. This works when the matrix is not diagonalisable. The key point here is that a matrix X can be decomposed into $X = U\Sigma V$, where U is the eigenvector of XX^T , V the eigenvector of $X^T X$, and Σ contains the square roots of the eigenvalues.

In summary

- PCA maximises variance through linear combinations of correlated features, providing X , orthogonal PCs
- PCA outputs uncorrelated principal components – so it can be used for decorrelating (or whitening), which is useful for ICA
- PCA requires standardisation before running
- PCA is unsupervised
- One more caveat on PCA - it is highly susceptible to outliers!!!

Caveats

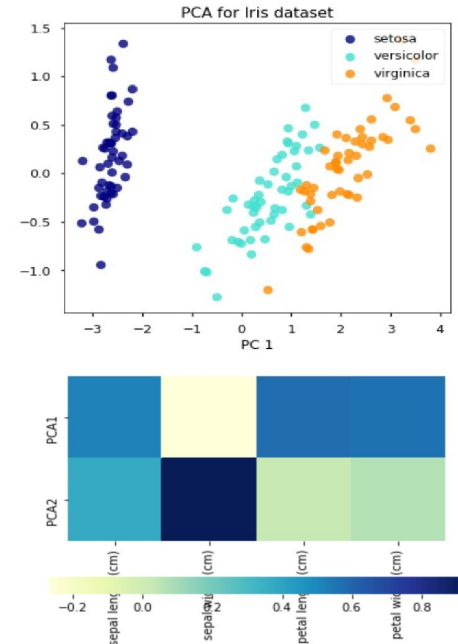
- PCA reads all data into memory at once – this won't work on massive datasets
- Use Randomised SVD to get an approximate decomposition. It is an iterative method, and the principal components are updated during every iteration
 - Note this is slightly slower
- If you run this on non-linear data it just won't find any separation

Okay enough theory show me the thing!

- Let's try applying this to a boring dataset like the iris one..
- This dataset has four columns representing the lengths and widths of the sepals and petals of three classes of flowers. Four dimensions is not something you can readily picture in your mind
- Also, we know from earlier that this is a linear dataset...
- So we can apply PCA!
- Don't forget to standardize the data first

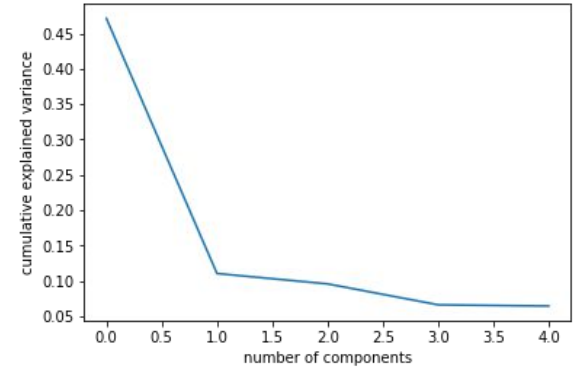
Okay enough theory show me the thing!

- The plot on the right is the result from using 2 PCs
- It shows the spread of the data between the two PCs – we can see that clearly, there is some difference between the classes that the PCs are finding!
- Using two PCs, we have captured 95% of the variance in the dataset – incredible
- The second plot shows us that the first PC (ie most of the variance) comes from the sepal length, and petal length and width, the second PC is mostly influenced by sepal width – this is an important finding, as it tells us that these are uncorrelated



Another PCA example - Boston

- We can also have a look at another well known dataset, the Boston housing prices dataset. The challenge with this dataset is to get the median value of homes, which is a real-valued, continuous variable.
- The covariance matrix for this dataset is the one shown earlier in the slides
- Now, when we want to plot things, we're only going to use the first 2 or 3 PCs, but we can still perform PCA and calculate more PCs (just that each subsequent one holds less variance)
 - So, we are interested in the cumulative variance as we add more PCs

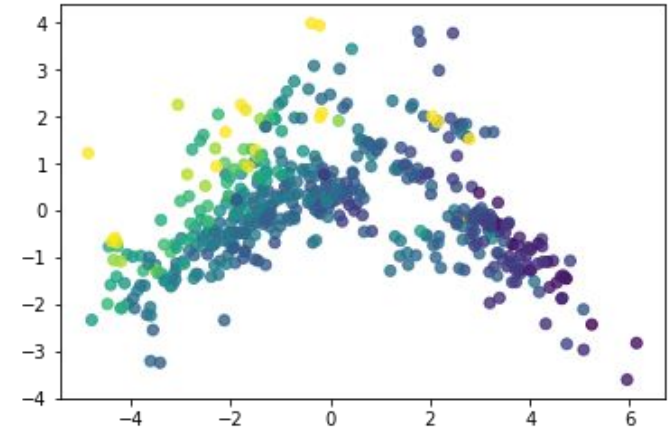


Here we can see that as we add components, we can explain more of the variance

- The first component captures 45%
- Two components only captures 58%
- Using 5 components we capture 80%

Another PCA example - Boston

- In the Iris example, we could plot the first 2 PCs and colour them according to their classes, here, we don't have classes...
- The plot *looks* pretty, but what does it even mean?!
- The colours represent a sliding scale of the median house prices (the target variable)
- So it looks like the first PC (x-axis), is separating high and low value houses, but the second PC is less useful
- We should bin the house prices and make up new target variables so that we can better understand this data – like we did for the Titanic dataset!



PCA for ML

- Remember one of the goals is to reduce the dimensionality to reduce processing time for training ML algorithms
- We can use the principal components as direct inputs
- If we perform PCA on the MNIST dataset (handwriting images), we can reduce the dimensionality from the current 784 pixels and compress it
- We can then use these PCs as inputs to something like a logistic regression model
- In the table here, I've shown how changing the number of PC components affects the accuracy and training time of a simple logistic regression classifier
- See notebook for more details
- We cut our training time in half, and actually improved our accuracy!!! Not bad going

Explained Variance	Training Time	Accuracy
50%	13.4 sec	90.1%
60%	14.9 sec	90.9%
70%	15.7 sec	91.7%
80%	16.5 sec	92.3%
95%	20.4 sec	92.1%
100%	31 sec	91.6%

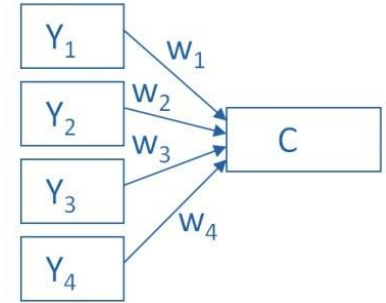
Why did the accuracy go *down* when we included more data?! I'm glad you asked! This is a great example of where there is some spurious signal in the data (noise) or extra correlations, that gets removed by PCA – so here is another win for PCA



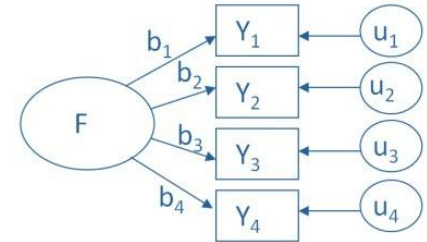
Factor Analysis

- Factor analysis (FA) and PCA are similar in many respects, but there is a fundamental difference between them: PCA is a linear combination of variables; Factor Analysis attempts to reconstruct or identify a latent variable. In other words, PCA is a linear combination of the observed variables, and FA the observed variables are linear combinations of unobserved, latent variables.
- To make this a bit more concrete, imagine we have some number of variables in our dataset, X . Our principal components will be calculated as:
 - $PC = w_1 * X_1 + w_2 * X_2 + \dots$
- Where w_i represent the weights.
- Whereas, in factor analysis, X is explained by the latent factor, F (or, we might assume multiple latent factors F):
 - $X_i = A_i * F + \epsilon_i$, (or, $X_i = A_{i1} * F_1 + \dots + A_{ij} * F_j$, if we assume multiple latent factors), A_i are the weights for each variable, and ϵ_i are error terms.
 - More compactly as $X = AF$, where A is a mixing matrix defining the relationship with the factors.
- The key part here is that each of the factors are uncorrelated.
- Conceptually FA and PCA are different:
 - Factor analysis assumes that there is some underlying variable (with a Gaussian distribution) that is not measured directly, but is causing the values of the observed variables to have their values.
 - FA explains the covariance in the data, as opposed to the maximal variance from PCA
 - From a mathematical perspective, another difference is that FA does not require orthogonality. Technically, PCA is a specific type of factor analysis.
- A big benefit to FA over PCA is that the outputs / factors are interpretable
- Because this finds latent variables, FA often gets used in interpreting things like survey data
- Similarly to PCA, FA produces loadings, which give a relative importance to each factor

PCA



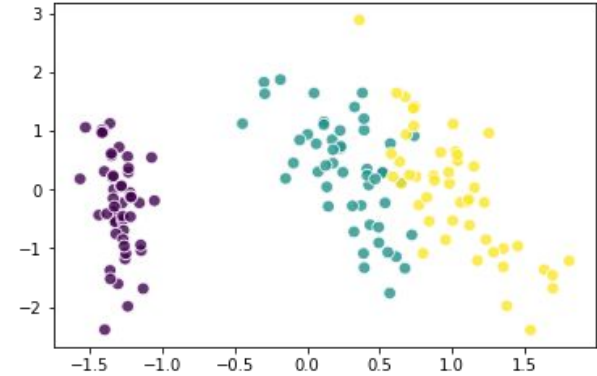
FA



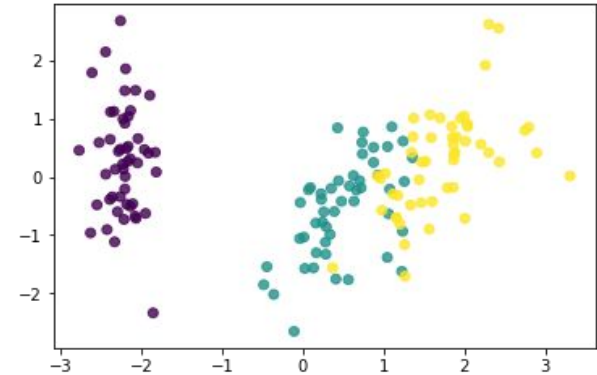
PCA vs FA

- So, when would we use FA over PCA? Or just FA in general?
- Run FA if we want to test a hypothesis that there are latent factors causing observed variables. This is often used in market research, social sciences and economics, where the answers might be indicative of some underlying feature. Imagine you wanted to get an understanding of anxiety - you can ask a bunch of questions to get this, but there is no easy way to get this out in a single question. Similarly, economic conditions might be caused by some unknown variable.
- FA can be used for dimensionality reduction, but rather than finding maximum variance, it's finding groups of variables that are correlated, regardless of variance.
- If we just want to reduce the dimensionality, or reduce the features into independent variables, then we use PCA.
- As an example, on the right is FA run on the iris dataset again. Compare these results to those of PCA below it. It's still identifying a separation in the data, but the values and shapes are different.

FA



PCA

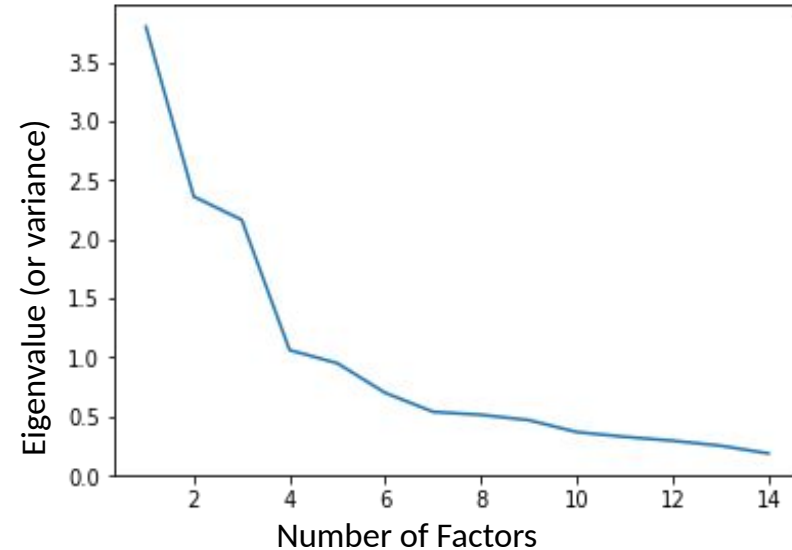


FA on survey data

- It's useful to see how we might apply this to some survey data too. We can have a quick look at airline passenger responses to how they rated different aspects of the airline experience.
- In the factor analysis, we want to look at the answers to the survey and if there is a common thread.
 - `survey_cols = ['Inflight wifi service', 'Departure/Arrival time convenient', 'Ease of Online booking', 'Gate location', 'Food and drink', 'Online boarding', 'Seat comfort', 'Inflight entertainment', 'On-board service', 'Leg room service', 'Baggage handling', 'Checkin service', 'Inflight service', 'Cleanliness']`
- So, we'll start off by reading in the data, setting up the factor analysis, and seeing how many factors we should use to describe the data

Survey factors

- On the right is a plot of the number of factors vs the explained variance
- Given this steep drop-off, we can probably decide on 3 or 4 factors. (The plot shows that the first factor has an eigenvalue of about 3.8, all the way to the fourth of 1.0)



Survey factors

- If we go with three factors, we get the following loadings of the factors (on the RHS)
- There are three columns representing the factors • The rows represent the survey answers • The important variables contributing to factor 1 (column one) are the 0.75, 0.79, 0.75 and 0.86, corresponding to "Food and Drink", "seat comfort", "Inflight entertainment" and "cleanliness".
- Do these four variables have anything in common - what do they represent?!
Hospitality? Comfort? That seems reasonable. This was not obvious from the original dataset or the questions asked - it is a latent variable.

		Factors		
Survey answers	[	0.16826952	0.12827119	0.75809134]
	[	-0.02950837	0.05968117	0.50138365]
	[	0.03023106	0.02091436	0.93277525]
	[	-0.0338282	-0.03231121	0.50404385]
	[	0.75263893	0.01094635	0.00616734]
	[	0.39545345	0.1138114	0.35906543]
	[	0.78999048	0.08146326	0.02725824]
	[	0.7456934	0.46674984	0.01203424]
	[	0.09388069	0.70115382	0.02900913]
	[	0.07445487	0.48144209	0.08065029]
	[	0.02346305	0.76474833	0.02769279]
	[	0.14351222	0.28418169	0.02888187]
	[	0.01813146	0.79977083	0.01825226]
	[	0.85842045	0.08814824	-0.00170807]]

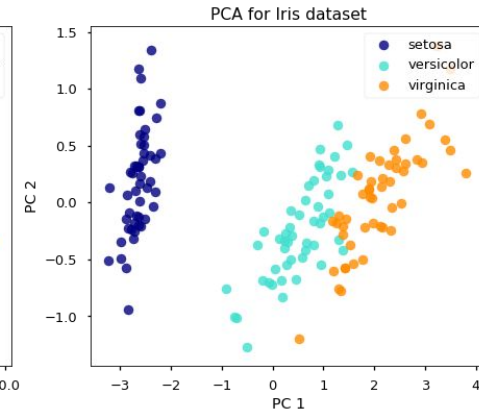
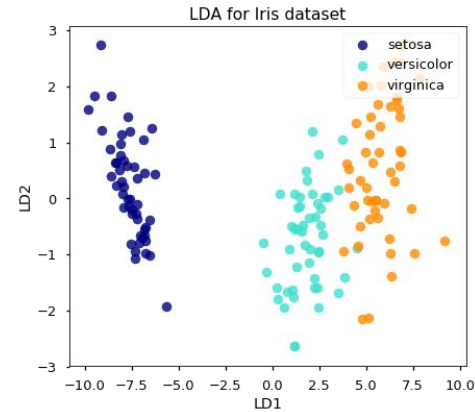
Linear Discriminant Analysis

- Linear Discriminant Analysis (LDA) is another linear transformation technique similar to FA and PCA
- It was developed as a classifier (in one of the previous notebooks, the one on diabetes, we used LDA!), but is commonly used for dimensionality reduction.
 - Go and check out the LDA example from that previous notebook – do you see why it is used?
- It is similar to PCA, but the key difference is that it is supervised. PCA maximises variance within the data, regardless of class labels. LDA tries to find the linear discriminants (the LDA equivalent of principal components) that maximises the separation between classes within the data.
- LDA outputs linear combinations of the input features, like PCA, but it does so by minimising a different object function. Unlike PCA, it is itself a classification technique, and should not be used as an input into a further regression problem.
- Note that for LDA you do not have to apply standardisation, although it doesn't hurt :) When using LDA for classification, the algorithm assumes that the data is normally distributed, statistically independent features, and that every class has identical covariance matrices. However, this is less of an issue if it is being used for dimensionality reduction (in particular, the statistical independence).
- Why would we use PCA on multiclass problems? LDA is built specifically to find differences in classes, as opposed to PCA looking at the data directly. Interestingly, if there are few samples per class, then PCA outperforms LDA. Although, in many cases, the two are used together - PCA is done to reduce the dimensionality, and the outputs are fed into an LDA.
- Many of the same shortcomings of PCA are apparent here - the data must be linear, if the data is too big, then it doesn't converge and we need to move onto other algorithms (or regularised versions of LDA).
- Again, the same metrics can be used to check how this is performing - scree plots, the eigenvalues, vectors, and the explained variance.



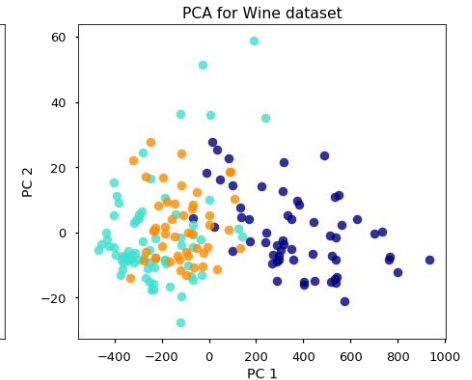
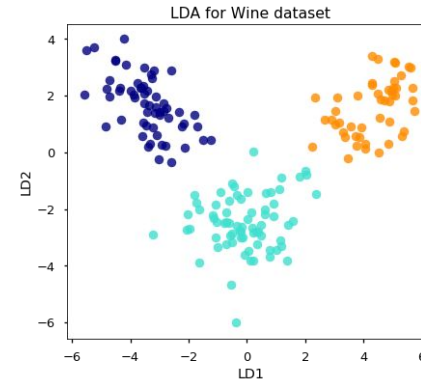
LDA vs PCA

- This short example highlights the difference quite well. PCA and LDA both are able to project the dataset into 2 components, but we can see from the plots that whilst PCA is finding maximal variance, LDA (specifically LD1) has found an axis that is separating the classes



LDA vs PCA

- Here is another example of LDA vs PCA on the wine dataset (<https://archive.ics.uci.edu/ml/datasets/wine>).
 - The wine dataset contains a bunch of attributes detailing the chemical analysis of wines from Italy. All wines were from the same region, but produced by different people. The goal is to figure out which chemical attributes come from which vineyard.
- How do we explain the extreme differences between the two results? PCA is finding the features with the most variance, but, the features with the most variance might actually be the same for each class.
 - (As an example, imagine we were looking at some sports teams and trying to predict performance based on the physical attributes of their players. Now, imagine that in this team, there are players ranging in height from 1.5 metres to 2 metres in every team, without any exceptions. Now, height might have the most variance over all, but when comparing teams, they are all exactly the same. PCA will focus on this, but it's actually useless for separating classes.



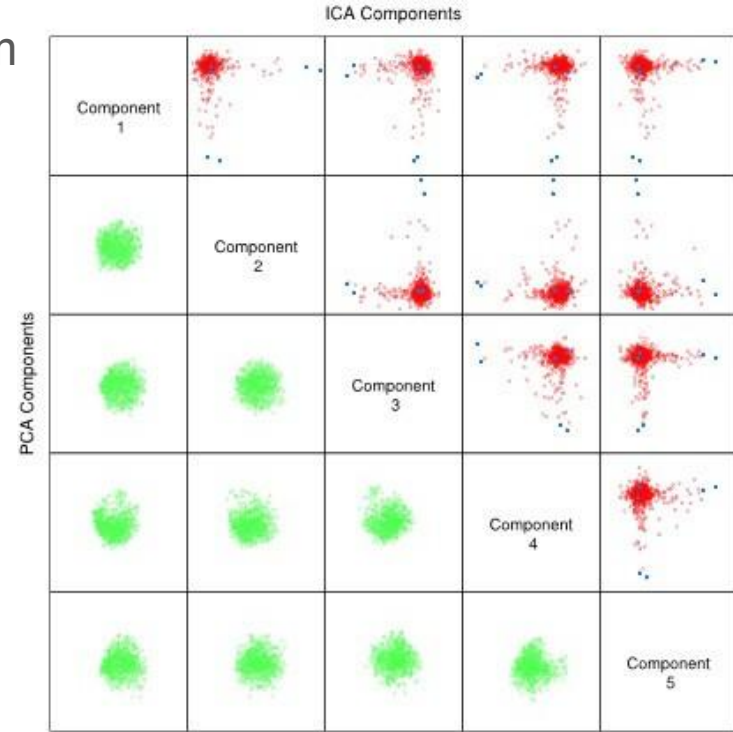
Independent component analysis

- Independent component analysis takes a multivariate signal and decomposes it into additive components by maximising the independence between each component
- Usually ICA is used for exactly that - separating superimposed signals (or blind source separation). However, it is also good for reducing dimensionality. It does a similar thing to PCA and FA in that it is breaking down a dataset into some lower dimensional basis, just using different criteria.
 - These sources can be images, audio signals, or time series data like EEGs
 - ICA has the same form as Factor Analysis, where each variable is assumed to be a linear combination of any number of unknown, latent variables, (remember $X=AF$, where A is a mixing matrix and F is the set of factors). The big difference is that the factors are no longer assumed to be uncorrelated, but statistically independent. This is important - if two variables are independent, that implies that there is no correlation. BUT no correlation does not imply that the variables are independent.
- Statistical independence criteria means we need to apply a whitening transformation to the data first to remove correlations (but note this is a necessary but not sufficient condition)
- One more key aspect of ICA, is the assumption that the factors are non-Gaussian. So, if the original latent factors are normally distributed, then ICA will not be effective.
- The three main assumptions, summarised:
 - Mixing process is linear
 - All source signals are independent of each other
 - All source signals are non-Gaussian



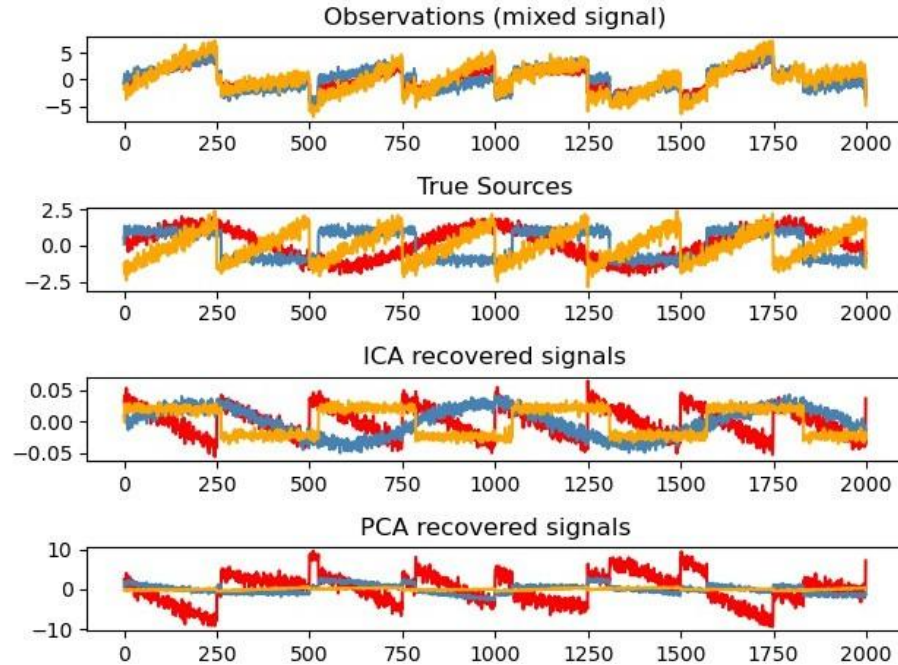
Independent component analysis

- Decomposition of a handwritten “3” from the MNIST dataset:
- See how the PCA components have a multivariate normal distribution?
- Whereas ICA components have long tails?



Blind source separation (cocktail party)

- This isn't really a dimensionality problem, but it is cool to see that it can do other things besides that!
- The cocktail party effect is the phenomena where someone can be in a noisy room, with many simultaneous conversations happening, yet be able to focus on only a single conversation and filter out the other conversations. The brain is able to separate the mixture of voices – this is what ICA does!
- There is a good example of this here:
 - https://scikit-learn.org/stable/auto_examples/decomposition/plot_ica_blind_source_separation.html



Summary

- Principal Component Analysis identifies the combination of attributes (principal components, or directions in the feature space) that are correlated that account for the most variance in the data (regardless of class) – unsupervised.
- Linear Discriminant Analysis tries to identify attributes that account for the most variance between classes. It is supervised.
- Factor Analysis identifies latent variables in the data by assuming that linear combinations of the features in the data are representing an unobserved phenomena and share covariance
- Independent Component Analysis identifies components in the data that are independent from each other, assuming that they have a non-Gaussian distribution. It is an unsupervised method too.